21.3 代表性扩展实现解析

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~820k tokens，输出 ~75k tokens（本章合计）

OpenClaw 的 `extensions/` 目录中包含了 **31 个**官方扩展，覆盖了企业通讯、开源协议、社交直播、区域性即时通讯等多种平台。本节选取具有代表性的扩展进行深入分析，展示不同类型平台的对接模式。

21.3.1 Microsoft Teams 扩展（Bot Framework 对接）

概览

MS Teams 扩展是最复杂的官方扩展之一，`extensions/msteams/src/` 包含 **51 个源文件**，涵盖了消息监听、发送、附件处理、投票、会话存储、Graph API 目录查询等功能。

特征	说明
通讯协议	Microsoft Bot Framework (REST Webhook)
认证方式	Azure AD App（appId + appPassword）
消息接收	Express HTTP 服务器监听 `/api/messages`
消息发送	Bot Framework SDK 的 `adapter.process()`
特色功能	Adaptive Cards、投票、会话持久化、Graph API 目录

入口注册

MS Teams 的入口极其简洁，遵循"薄入口"原则：

```
// extensions/msteams/index.ts
import type { OpenClawPluginApi } from "openclaw/plugin-sdk";
import { msteamsPlugin } from "./src/channel.js";
import { setMSTeamsRuntime } from "./src/runtime.js";

const plugin = {
  id: "msteams",
  configSchema: emptyPluginConfigSchema(),
  register(api: OpenClawPluginApi) {
    setMSTeamsRuntime(api.runtime);           // 缓存运行时引用
    api.registerChannel({ plugin: msteamsPlugin }); // 注册通道插件
  },
};
export default plugin;
```

注意 `setMSTeamsRuntime(api.runtime)` 这个模式——由于插件模块无法在顶层访问 `PluginRuntime`，需要在 `register()` 时将其缓存到模块级变量中，供后续的 `monitor.ts`、`send.ts` 等模块使用。这是所有扩展共用的惯用模式。

通道插件定义

`msteamsPlugin` 是一个完整的 `ChannelPlugin` 实现，包含了第 11 章介绍的多数适配器槽位：

```typescript
// extensions/msteams/src/channel.ts（关键结构）
export const msteamsPlugin: ChannelPlugin<ResolvedMSTeamsAccount> = {
  id: "msteams",
  meta: {
    label: "Microsoft Teams",
    selectionLabel: "Microsoft Teams (Bot Framework)",
    aliases: ["teams"],
    order: 60,
  },

  // 能力声明
  capabilities: {
    chatTypes: ["direct", "channel", "thread"],
    polls: true,
    threads: true,
    media: true,
  },

  // 适配器实现
  config: { /* 账号管理 */ },
  security: { /* 安全策略 */ },
  pairing: { /* 配对审批 */ },
  messaging: { /* 消息目标解析 */ },
  directory: { /* 通讯录查询 */ },
  resolver: { /* 目标ID解析 */ },
  actions: { /* Adaptive Card 发送 */ },
  outbound: msteamsOutbound,
  status: { /* 状态监控 */ },
  gateway: { /* 启动消息监听 */ },
  threading: { /* 线程上下文 */ },
  groups: { /* 群组工具策略 */ },
  onboarding: msteamsOnboardingAdapter,
  agentPrompt: { /* Agent 工具提示 */ },
};
```

消息监听：Express + Bot Framework

MS Teams 最独特的地方是它使用 **Webhook 推送**模式，而非长轮询。OpenClaw 为此启动了一个 Express HTTP 服务器：

```ts
// extensions/msteams/src/monitor.ts（简化）
export async function monitorMSTeamsProvider(opts) {
  const port = msteamsCfg.webhook?.port ?? 3978;
  const express = await import("express");          // 动态导入
  const { sdk, authConfig } = await loadMSTeamsSdkWithAuth(creds);

  // 创建 Bot Framework Adapter
  const adapter = createMSTeamsAdapter(authConfig, sdk);

  // 注册消息处理
  const handler = registerMSTeamsHandlers(new ActivityHandler(), {
    cfg, runtime, adapter, textLimit, ...
  });

  // 启动 Express 服务器
  const expressApp = express.default();
  expressApp.use(express.json());
  expressApp.use(authorizeJWT(authConfig));          // JWT 验证中间件
  expressApp.post("/api/messages", (req, res) => {
    adapter.process(req, res, (context) => handler.run(context));
  });

  const httpServer = expressApp.listen(port);
  return { app: expressApp, shutdown: () => httpServer.close() };
}
```

> **衍生解释**：**Bot Framework** 是微软提供的机器人开发框架。与 Telegram Bot API 类似，开发者注册一个 Bot，配置 Webhook URL，微软服务器会将用户消息以 HTTP POST 推送到该 URL。`ActivityHandler` 是 Bot Framework SDK 中处理消息活动（Activity）的基类，OpenClaw 通过继承它来处理消息、事件等不同类型的 Activity。

Adaptive Cards 支持

MS Teams 的一个亮点是支持 **Adaptive Cards**——一种声明式的富内容卡片格式：

```
// extensions/msteams/src/channel.ts — actions 适配器
actions: {
  supportsCards: ({ cfg }) => Boolean(resolveMSTeamsCredentials(cfg.channels?.msteams))
  handleAction: async (ctx) => {
    if (ctx.action === "send" && ctx.params.card) {
      const result = await sendAdaptiveCardMSTeams({
        cfg: ctx.cfg,
        to: ctx.params.to,
        card: ctx.params.card,
      });
      return { content: [{ type: "text", text: JSON.stringify({ ok: true, ...result })
    }
    return null;   // 非卡片操作交给默认处理
  },
},
```

21.3.2 Matrix 扩展（matrix-js-sdk 对接）

概览

Matrix 是一个**去中心化**的开源通讯协议，与 MS Teams 的中心化模型形成鲜明对比。

特征	说明
通讯协议	Matrix Client-Server API (REST + Server-Sent Events)
认证方式	Homeserver URL + Access Token / Password
消息接收	matrix-js-sdk 的 `/sync` 长轮询
消息发送	Matrix Client API (`PUT /_matrix/client/v3/rooms/{roomId}/send/...`)
特色功能	端到端加密（E2EE via matrix-sdk-crypto-nodejs）、多 Homeserver

入口注册

Matrix 扩展的入口与 MS Teams 结构完全一致：

```
// extensions/matrix/index.ts
const plugin = {
  id: "matrix",
  configSchema: emptyPluginConfigSchema(),
  register(api: OpenClawPluginApi) {
    setMatrixRuntime(api.runtime);
    api.registerChannel({ plugin: matrixPlugin });
  },
};
```

开源协议的特殊处理

Matrix 扩展的 `channel.ts` 比 MS Teams 更关注**去中心化特性**：

```
// extensions/matrix/src/channel.ts — 账号支持多 Homeserver
config: {
  listAccountIds: (cfg) => listMatrixAccountIds(cfg),  // 可能有多个账号
  resolveAccount: (cfg, accountId) => resolveMatrixAccount({ cfg, accountId }),
  deleteAccount: ({ cfg, accountId }) =>
    deleteAccountFromConfigSection({
      cfg, sectionKey: "matrix", accountId,
      clearBaseFields: ["homeserver", "userId", "accessToken", "password", "deviceName"
    }),
},

// 安全策略：Matrix 群组默认使用 allowlist 策略
security: {
  resolveDmPolicy: ({ account }) => ({
    policy: account.config.dm?.policy ?? "pairing",  // 默认配对模式
    allowFrom: account.config.dm?.allowFrom ?? [],
  }),
},
```

Matrix 的目标解析（Target Resolution）需要处理多种 ID 格式：

```
messaging: {
  targetResolver: {
    looksLikeId: (raw) => {
      const trimmed = raw.trim();
      if (/^(matrix:)?[!#@]/i.test(trimmed)) return true;  // !roomId, #alias, @userId
      return trimmed.includes(":");                          // server:port 格式
    },
    hint: "<room|alias|user>",
  },
},
```

> **衍生解释**：Matrix 的 ID 系统使用 sigil（前缀符号）区分不同实体：`@` 表示用户（如 `@alice:matrix.org`），`!` 表示房间（如 `!abc123:matrix.org`），`#` 表示房间别名（如 `#general:matrix.org`）。冒号后面是 Homeserver 地址。这种去中心化 ID 设计让用户可以在不同服务器上拥有身份。

端到端加密

Matrix 扩展通过 `matrix-sdk-crypto-nodejs` 支持 E2EE（端到端加密）。这是一个 **Native Node 模块**（Rust 编译），这正是它被放在扩展而非核心通道中的主要原因——native 模块的编译可能在某些平台上失败，放在扩展中可以避免影响主包安装。

21.3.3 Twitch 扩展

概览

Twitch 是一个直播平台，其聊天系统基于 **IRC（Internet Relay Chat）** 协议。

特征	说明
通讯协议	IRC over WebSocket (tmi.js)
认证方式	OAuth Token
消息接收	IRC WebSocket 实时推送
消息发送	IRC PRIVMSG 命令
源文件数	28 个（含测试）

入口注册

```
// extensions/twitch/index.ts
const plugin = {
  id: "twitch",
  configSchema: emptyPluginConfigSchema(),
  register(api: OpenClawPluginApi) {
    setTwitchRuntime(api.runtime);
    api.registerChannel({ plugin: twitchPlugin });
  },
};
```

IRC 协议的特殊性

与基于 HTTP 的 MS Teams 和基于 REST API 的 Matrix 不同，Twitch 使用的是 **IRC 协议**。这意味着：

1. **无线程概念**：IRC 是扁平的消息流，没有回复链或线程
2. **消息长度限制严格**：IRC 单条消息限制为 500 字节
3. **无媒体支持**：IRC 只支持纯文本
4. **实时性极高**：基于 WebSocket 的持久连接

> **衍生解释**：**IRC（Internet Relay Chat）** 是 1988 年诞生的实时聊天协议，是互联网最古老的聊天协议之一。虽然已经被现代即时通讯软件取代，但 Twitch 为了支持直播间的大规模实时聊天，沿用了 IRC 协议（经过扩展）。IRC 的核心概念是"频道"（Channel）和"消息"（PRIVMSG），消息通过持久 TCP 连接双向推送。

Twitch 扩展的源码结构反映了这些约束——它有 `access-control.ts` （频道权限管理）、 `client-manager-registry.ts` （多频道连接管理）、但没有 threading 或 media 相关的模块。

21.3.4 Nostr 扩展

概览

Nostr 是一个**去中心化社交协议**，与 Matrix 类似但更加极简。

特征	说明
通讯协议	Nostr NIP-04（加密直接消息）

特征	说明
认证方式	公私钥对（Ed25519/Secp256k1）
消息接收	通过中继（Relay）订阅事件
消息发送	签名后发布到中继
特色功能	HTTP Profile 管理、NIP-05 验证

入口注册：超越单纯的通道注册

Nostr 扩展展示了一个更复杂的注册模式——除了注册通道，还注册了 HTTP 处理器：

```
// extensions/nostr/index.ts（简化）
register(api: OpenClawPluginApi) {
  setNostrRuntime(api.runtime);
  api.registerChannel({ plugin: nostrPlugin });

  // 注册 HTTP 处理器：提供 Nostr Profile 管理 API
  const httpHandler = createNostrProfileHttpHandler({
    getConfigProfile: (accountId) => {
      const cfg = getNostrRuntime().config.loadConfig();
      return resolveNostrAccount({ cfg, accountId }).profile;
    },
    updateConfigProfile: async (accountId, profile) => {
      const cfg = getNostrRuntime().config.loadConfig();
      await getNostrRuntime().config.writeConfigFile({
        ...cfg,
        channels: { ...cfg.channels, nostr: { ...cfg.channels.nostr, profile } },
      });
    },
    getAccountInfo: (accountId) => {
      // 返回公钥和中继列表
    },
  });
  api.registerHttpHandler(httpHandler);
}
```

这说明插件的注册能力不仅限于通道——它可以同时注册 HTTP 端点，为 Web UI 或外部工具提供管理接口。

> **衍生解释：Nostr（Notes and Other Stuff Transmitted by Relays）** 是一个极简的去中心化协议。与 Matrix 不同，Nostr 没有服务器账号的概念——用户的身份就是一对密钥。消息（称为 Event）用私钥签名后发布到多个 Relay（中继服务器），接收者从 Relay 订阅感兴趣的消息。NIP-04 是 Nostr 的加密直接消息规范。

核心组件

Nostr 扩展的 `src/` 目录中有几个值得注意的模块：

- **`nostr-bus.ts`**：消息总线，管理与多个 Relay 的 WebSocket 连接
- **`nostr-state-store.ts`**：状态持久化，记录已处理的事件 ID
- **`seen-tracker.ts`**：事件去重，避免从多个 Relay 收到同一消息时重复处理
- **`nostr-profile.ts`**：Nostr Profile（NIP-05）管理

21.3.5 Google Chat 扩展

概览

Google Chat 使用 **Webhook 推送**模式，与 MS Teams 类似。

特征	说明
通讯协议	Google Chat API (HTTP Webhook)
认证方式	Google Cloud Service Account
消息接收	HTTP Webhook 推送
消息发送	Google Chat REST API

入口注册：Dock + HTTP Handler

Google Chat 的注册展示了另一种模式——同时注册了 `dock` 和 `httpHandler`：

```
// extensions/googlechat/index.ts
register(api: OpenClawPluginApi) {
  setGoogleChatRuntime(api.runtime);
  api.registerChannel({ plugin: googlechatPlugin, dock: googlechatDock });
  api.registerHttpHandler(handleGoogleChatWebhookRequest);
}
```

`dock` 是第 11 章介绍的 `ChannelDock`，它提供了通道级别的消息处理管道。`httpHandler` 直接处理 Google 发来的 Webhook 请求。这种分离设计使得 Webhook 处理和消息管道可以独立演进。

21.3.6 Zalo / Zalo Personal 扩展

概览

Zalo 是越南最大的即时通讯平台，OpenClaw 提供了**两个** Zalo 扩展：

扩展	协议	使用场景
`zalo` （Bot API）	Zalo Official Account API	企业客服场景
`zalouser` （个人号）	zca-cli（逆向工程）	个人账号自动化

Zalo Bot API

```
// extensions/zalo/index.ts
register(api: OpenClawPluginApi) {
  setZaloRuntime(api.runtime);
  api.registerChannel({ plugin: zaloPlugin, dock: zaloDock });
  api.registerHttpHandler(handleZaloWebhookRequest);  // Webhook 接收
}
```

结构与 Google Chat 非常相似——Webhook 接收 + REST API 发送。

Zalo Personal：工具注册

Zalo Personal 扩展展示了一个独特的模式——除了注册通道，还注册了一个 **Agent 工具**：

```
// extensions/zalouser/index.ts
register(api: OpenClawPluginApi) {
  setZalouserRuntime(api.runtime);
  api.registerChannel({ plugin: zalouserPlugin, dock: zalouserDock });

  // 注册 Agent 工具——让 AI 可以主动操作 Zalo
  api.registerTool({
    name: "zalouser",
    label: "Zalo Personal",
    description: "Send messages and access data via Zalo personal account. " +
      "Actions: send, image, link, friends, groups, me, status.",
    parameters: ZalouserToolSchema,
    execute: executeZalouserTool,
  });
}
```

这意味着当用户通过 Zalo Personal 与 AI 对话时，AI 不仅能回复消息，还能通过工具调用主动发送消息、查询好友列表、检查认证状态等——这是纯通道注册无法实现的能力。

21.3.7 飞书（Feishu/Lark）扩展

概览

飞书扩展是**功能最丰富**的官方扩展， `extensions/feishu/src/` 包含 30 个源文件。

特征	说明
通讯协议	飞书开放平台 API
认证方式	App ID + App Secret
消息接收	飞书事件订阅（HTTP Webhook）
特色功能	多维表格、云文档、知识库、云盘、权限管理

入口注册：最丰富的注册模式

飞书扩展展示了最复杂的注册模式——除了通道，还注册了 **5 组 Agent 工具**：

```
// extensions/feishu/index.ts
register(api: OpenClawPluginApi) {
  setFeishuRuntime(api.runtime);
  api.registerChannel({ plugin: feishuPlugin });

  // 注册飞书云文档工具
  registerFeishuDocTools(api);    // 创建/编辑飞书文档
  registerFeishuWikiTools(api);   // 操作知识库
  registerFeishuDriveTools(api);  // 云盘文件操作
  registerFeishuPermTools(api);   // 权限管理
  registerFeishuBitableTools(api); // 多维表格操作
}
```

这使得 AI Agent 在飞书通道中不仅能聊天，还能直接操作飞书的办公套件——创建文档、查询知识库、管理文件权限、读写多维表格。这是 OpenClaw 插件系统设计哲学的完美体现：**通道不仅是消息管道，更是能力入口**。

源码模块概览

```
extensions/feishu/src/
├── channel.ts          # ChannelPlugin 定义
├── monitor.ts          # 事件订阅监听
├── send.ts             # 消息/卡片发送
├── media.ts            # 图片/文件上传
├── mention.ts          # @提及解析
├── reactions.ts        # 表情回复
├── typing.ts           # 正在输入状态
├── docx.ts             # 云文档工具
├── wiki.ts             # 知识库工具
├── drive.ts            # 云盘工具
├── perm.ts             # 权限管理工具
├── bitable.ts          # 多维表格工具
├── doc-schema.ts       # 文档工具 Schema
├── wiki-schema.ts      # 知识库工具 Schema
├── drive-schema.ts     # 云盘工具 Schema
├── perm-schema.ts      # 权限工具 Schema
└── tools-config.ts     # 工具配置
```

飞书还在 `skills/` 目录中提供了预设的 Agent Skill（技能），进一步增强 AI 在飞书场景中的能力。

21.3.8 LINE 扩展

概览

LINE 是日本和东南亚最流行的即时通讯应用。

特征	说明
通讯协议	LINE Messaging API (HTTP Webhook + REST)
认证方式	Channel Access Token
消息接收	Webhook 推送
特色功能	Flex Message（富内容模板）、Markdown 转 LINE 格式

入口注册：命令注册

```
// extensions/line/index.ts
register(api: OpenClawPluginApi) {
  setLineRuntime(api.runtime);
  api.registerChannel({ plugin: linePlugin });
  registerLineCardCommand(api);  // 注册 /card 命令
}
```

LINE 扩展使用了 `registerCommand()` 来注册自定义命令——用户可以发送 `/card` 来发送 LINE Flex Message 卡片。

Flex Message 与 Markdown 转换

LINE 扩展最独特的技术特征是 **Markdown → LINE Flex Message** 的转换。由于 LINE 不支持 Markdown，扩展提供了一套完整的转换管道：

```
// 通过 plugin-sdk 导出的 LINE 工具
export { processLineMessage, hasMarkdownToConvert, stripMarkdown } from "../line/markdo
export { createInfoCard, createListCard, createImageCard,
         createActionCard, createReceiptCard } from "../line/flex-templates.js";
```

这些函数将 Agent 生成的 Markdown 文本转换为 LINE 原生的 Flex Message JSON 结构，在移动端渲染为美观的卡片式消息。

扩展模式总结

通过分析 8 个代表性扩展，我们可以归纳出几种典型的注册模式：

模式	代表	注册内容
纯通道	MS Teams, Matrix, Twitch	`registerChannel()`
通道 + Dock	Google Chat, Zalo	`registerChannel({ dock })`
通道 + HTTP	Nostr, Google Chat	`registerChannel() + registerHttpHandler()`
通道 + 工具	Zalo Personal, 飞书	`registerChannel() + registerTool()`
通道 + 命令	LINE	`registerChannel() + registerCommand()`
全功能	飞书	通道 + 5 组工具 + 技能

所有扩展共享一个固定的入口模式：

```
const plugin = {
  id: "...",
  configSchema: emptyPluginConfigSchema(),
  register(api: OpenClawPluginApi) {
    setXxxRuntime(api.runtime);      // 1. 缓存运行时
    api.registerChannel({ ... });     // 2. 注册通道
    // 3. 可选：注册工具/命令/HTTP/服务
  },
};
export default plugin;
```

本节小结

1. **MS Teams 扩展**是 Webhook 推送模式的代表，通过 Express HTTP 服务器接收 Bot Framework 消息，支持 Adaptive Cards 富内容卡片。
2. **Matrix 扩展**展示了去中心化协议的对接，需要处理多 Homeserver、sigil ID 格式、E2EE 加密等特殊需求，native 依赖是它被放在扩展中的主要原因。
3. **Twitch 扩展**基于 IRC 协议，是所有扩展中最简约的——无线程、无媒体、纯文本。

4. **Nostr 扩展**除了通道注册，还通过 `registerHttpHandler()` 暴露了 Profile 管理 API，展示了插件的多维注册能力。
5. **飞书扩展是功能最丰富的官方扩展**，除了通道还注册了云文档、知识库、云盘、权限、多维表格五组 Agent 工具，体现了"通道即能力入口"的设计哲学。
6. **所有扩展共享固定的入口模式**：缓存 Runtime → 注册通道 → 可选注册附加能力。配置通过 `openclaw.plugin.json` 清单声明，插件 ID 全局唯一。